

LAB SESSION 4

Programming Exercise:

Question 1: Write a function swap that takes two integer pointers as arguments and swaps the values they point to.

```
C:\C++ PROGRAMS > g++ tut1.cpp & main()
1  #include <iostream>
2  using namespace std;
3
4  // Function to swap two integers using pointers
5  void swap(int *a, int *b)
6  {
7      int temp = *a; // Store the value of a in temp
8      *a = *b;       // Copy the value of b into a
9      *b = temp;     // Copy the value of temp into b
10 }
11
12 int main()
13 {
14     int x = 10;
15     int y = 20;
16
17     cout << "Before swapping:\n";
18     cout << "x = " << x << ", y = " << y << endl;
19
20     // Call the swap function with addresses
21     swap(&x, &y);
22
23     cout << "After swapping:\n";
24     cout << "x = " << x << ", y = " << y << endl;
25
26     return 0;
}
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\"; if ($?) { g++ tut1.cpp -o tut1 } ;
if ($?) { .\tut1 }
Before swapping:
x = 10, y = 20
After swapping:
x = 20, y = 10
PS C:\C++ PROGRAMS>
```

Question 2: Write a function reverse Array that takes an integer array and its size as arguments and reverses the elements of the array using pointers.

```
C:\C++ PROGRAMS > g++ tut1.cpp ...
5  void reverseArray(int *arr, int size)
6  {
7      while (start < end)
8      {
9          int temp = *start;
10         *start = *end;
11         *end = temp;
12
13         start++; // Move forward
14         end--;   // Move backward
15     }
16 }
17
18 int main()
19 {
20     int arr[] = {10, 20, 30, 40, 50};
21     int size = sizeof(arr) / sizeof(arr[0]);
22
23     cout << "Original array: ";
24     for (int i = 0; i < size; i++)
25         cout << arr[i] << " ";
26
27     reverseArray(arr, size); // Call function to reverse
28
29     cout << "\nReversed array: ";
30     for (int i = 0; i < size; i++)
31         cout << arr[i] << " ";
32
33     return 0;
34 }
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\"; if ($?) { g++ tut1.cpp -o tut1 } ;
if ($?) { .\tut1 }
Original array: 10 20 30 40 50
Reversed array: 50 40 30 20 10
PS C:\C++ PROGRAMS>
```

Question 3: Write function find Max that takes an integer array and its size as arguments and returns a pointer to the maximum element in the array.

```
C:\C++ PROGRAMS > C- tut1.cpp > (main)
1  #include <iostream>
2  using namespace std;
3
4  // Function to find the maximum element and return a pointer to it
5  int *findMax(int *arr, int size)
6  {
7      if (size == 0)
8          return nullptr; // Edge case: empty array
9
10     int *maxPtr = arr; // Assume first element is max
11
12     for (int i = 1; i < size; i++)
13     {
14         if (*(arr + i) > *maxPtr)
15         {
16             maxPtr = arr + i; // Update maxPtr if current is greater
17         }
18     }
19     return maxPtr;
20 }
21 int main()
22 {
23     int arr[] = {15, 30, 22, 5, 90, 45};
24     int size = sizeof(arr) / sizeof(arr[0]);
25
26     int *maxElement = findMax(arr, size);
27
28     if (maxElement != nullptr)
29         cout << "Maximum element: " << *maxElement << endl;
30     else
31         cout << "Array is empty." << endl;
32
33     return 0;
34 }
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Maximum element: 90
PS C:\C++ PROGRAMS>
```

Question 4: Write a function remove Duplicates that takes an integer array and its size as arguments and removes duplicates, keeping only the first occurrence of each element.

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Original array: 10 20 10 30 20 40 50 30
Array after removing duplicates: 10 20 30 40 50
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Function to remove duplicates in-place
void removeDuplicates(int* arr, int& size) {
    int newSize = 0;

    for (int i = 0; i < size; i++) {
        bool isDuplicate = false;

        // Check if arr[i] already exists in arr[0..newSize-1]
        for (int j = 0; j < newSize; j++) {
            if (arr[i] == arr[j]) {
                isDuplicate = true;
                break;
            }
        }

        // If not duplicate, add to the result position
        if (!isDuplicate) {
            arr[newSize] = arr[i];
            newSize++;
        }
    }

    size = newSize; // Update the original size
}

int main() {
    int arr[] = {10, 20, 10, 30, 20, 40, 50, 30};
    int size = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";

    removeDuplicates(arr, size); // Remove duplicates

    cout << "\nArray after removing duplicates: ";
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Question 5: Write a function isPalindrome that takes a string as an argument and returns true if the string is a palindrome, false otherwise. Use pointers to check for palindrome.

```
C:\C++ PROGRAMS > C++ tut1.cpp > ...
1  #include <iostream>
2  #include <cstring> // For strlen
3  using namespace std;
4
5  // Function to check if a string is a palindrome using pointers
6  bool isPalindrome(const char *str)
7  {
8      const char *start = str;           // Pointer to first character
9      const char *end = str + strlen(str) - 1; // Pointer to last character
10
11      while (start < end)
12      {
13          if (*start != *end)
14              return false; // Not a palindrome
15
16          start++;
17          end--;
18      }
19
20      return true; // All matched
21  }
22
23  int main()
24  {
25      const char *word = "madam";
26
27      if (isPalindrome(word))
28          cout << word << " is a palindrome." << endl;
29      else
30          cout << word << " is not a palindrome." << endl;
31
32      return 0;
33  }
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
madam is a palindrome.
PS C:\C++ PROGRAMS>
```